1. Introduction

Languages generated by Large Language Models (LLMs) closely resemble those of human beings yet they still have a tendency to "hallucinate", meaning to generate text that, although grammatically correct, is factually wrong or not supported at all. A thorough overview of available detection methods points to the distinction between two main aspects of the problem: factuality hallucination, when the generated content is inconsistent with the verifiable world knowledge, and faithfulness hallucination when output drifts from the user original intention the contextual information provided, or its logical internal consistency, [1]. The magnitude of the issue is quite huge: an analysis of hallucination datasets collected automatically has come up with an estimate that open- and closed-source models hallucinate about 20% to 30% of all responses. Furthermore, it depends on the topic and on the specific model, respectively [12]. The reason why hallucinated pieces of text are hard to detect is because they sound natural and confident, so one cannot detect it by reading alone and that is why there has been a massive increase of research work in the area of hallucination detection [1].

Shared evaluation efforts have guided the way detection systems have been developed and compared. The SemEval-2024 Task6 shared task (SHROOM) set a common benchmark in identifying model-agnostic and model-aware hallucinations through a simple yes/no (fluent overgeneration) classification.

The various approaches revealed by participating in the task include: methods that measure output similarities by matching against target text reference, using pre-trained embedding models like LaBSE, T5, and DistilUSE [5], and exploring the degree to which those embeddings can reveal content that has been hallucinated [6]; model-agnostic methods where word alignment plus fine-tuned BERT-based classifiers are the main features, and model-aware approaches where attention values and layer activations from the LLM's internal data are tapped. When the two are joined the accuracy rate hits above 80% [7]. Besides classification, hallucination detection has moved into generation tasks as well: in machine translation, hallucinations have emerged when a translation gets rid of its source completely and this detachment is measured by source contribution. This source contribution, together with cross-lingual sentence similarity detection, has improved detection accuracy a lot [4].

Rather than focusing on the hallucinations that result in the text itself, there is also a line of work that detects hallucinations from the model itself. INSIDE demonstrates that it is possible to leverage the internal working of LLM to detect hallucinations, by measuring semantic diversity via EigenScores of the responses and limiting the extreme activation level which reduces overconfident generation of the model [9]. LLM-Check goes a step further and develops closed-model settings by analysing the eigenvectors of hidden-state representations and quantifying the uncertainty of generating output tokens, and in this way, it captures the heterogeneity of how hallucinations are manifested in different domains [10]. Methods of self-consistency, which verify if independently generated answers are compatible with each other, are efficient but also time-consuming as they involve multiple runs of the system; Decoding Memory Pipeline mitigates this by harnessing the shared parts of the prefix tokens across the generations without any degradation of detection performance even achieving up to a threefold increase in speed [11].

Concerning the data, AutoHall offers a model-agnostic method of the hallucination data preparation by using fact-checking resources that are available to us; furthermore, AutoHall develops a detector that works without any training data and is a kind of blackbox that relies on self-contradiction; it also shows the effectiveness of this new approach in terms of accuracy over the data set [12].

Applied systems are the practical implementation of these methods. One example is a real-time system to detect hallucinations in LLMs that merges multiple techniques: retrieving evidence from trusted sources, semantic similarity over sentence embeddings, self-consistency across multiple responses, and a final decision module that classifies the output as factual or hallucinated [2]. The effectiveness of such an approach is consistent with another well-known pattern a single method for hallucination detection leaves blind spots, whereas combining different approaches gets the best results [2]。
Still, certified analysis of recent results warns us against over-reliance on the detection methods based on embedding and NLI. It turns out that the embedding- and NLI-based methods do very well on hallucinations created to look like real but are actually synthetic samples, yet they fail completely when real hallucinating examples from RLHF-aligned models are presented to them. Indeed, the hardest cases of these are almost semantically different from the faithful ones. This kind of situation was defined as "Semantic Illusion" [13]. That study gives us a direct reason to design verification pipelines that are not only based on embedding similarity but also involve other means.

The paper suggests a twelve-step system for hallucination-checking and fact verification of RAG outputs, integrating the strengths of different techniques while addressing the gaps. Instead of only performing a final check, the verification process is turned into a multi-stage, open process, with a few stages. First, a query is transformed into a new form, which is enriched by terms of interest using zero-shot Named Entity Recognition (NER) methods (GLiNER, spaCy) and a instruction-tuned Llama 3.1 8B generation module (Qwen3-8B as a backup); the database of found passages is the next searched via a hybrid, dense-plus-sparse retrieval scheme (BGE-M3+FAISS, along with sparse BM25) combined by reciprocal rank fusion, and after that an cross-encoder reranking is done(BGE-reranker-v2-m3). To each statement of the model reply, the method of evidence is done with a multilingual NLI model (mDeBERTa-v3, XNLI), in the way of NLI verifiers that were best in the shared tasks described in [8]. In parallel, a confidence rating estimator on the signals verification (Platt scaling and temperature scaling) offers the scores that are calibrated to uncertainty-based detection [4], [10], and a regeneration action takes over the rewriting of all the claims from the response that failed verification, which introduces the concept of feedback from the detection phase to the countermeasure phase

The approach of the pipeline is to avoid relying solely on vector similarity as a method for proving correctness because it only provides a factuality signal at best and is, therefore, by no means a reliable one [13]. In our case, the pipeline relies primarily on facts retrieved from the database and judgment based on Natural Language Inference (NLI) decisions sentence-by-sentence. The final judgment along with the level of certainty, the verdict on each claim, and a summary of all thirteen phases of the process are provided together. In fact, by default, every component of the pipeline uses real HuggingFace checkpoints, and for each of the thirteen steps it is a separate, traceable module that also records its own evidence and decisions. The backend for exposing the system is through FastAPI, and the Flask front-end allows users to look at everything as it happens. Finally, the system was evaluated on HaluEval and TruthfulQA benchmarks in addition to using generation metrics(BERTScore, ROUGE-L) together with detection metrics.

2. Related Works

2.1 Taxonomies and Benchmarks

Bestetti et al. [1] provide a well-organized overview of hallucination detection tools, distinguishing two types of hallucination: factuality hallucination where generated text contradicts what is a common world knowledge and faithfulness hallucination, which is output generated text being in contradiction with input or with the context.
The SHROOM shared task [3] has introduced detection as a challenge with two main tracks, model-agnostic and modelAware, of binary classification of overgeneration errors, thereby allowing research community to have a uniform benchmark.

2.2 Sentence-Similarity Detection

Dale et al. have developed a method to identify machine translation hallucinations that are translations not closely connected with the source, cross-lingual sentence similarity helped them to increase accuracy.
Iordanidou et al. investigated various pre-trained encoders (LaBSE, T5, DistilUSE) when distinguishing between hallucinations and factual claims, while Bendahman et al. demonstrate how these embeddings highly react to hallucinations.
Score calculation through embedding is very straightforward and quite efficient yet the results rely solely upon the encoder, that means, it is highly subjective.

2.3 Model-Aware Detection

The method INSIDE [9] consists of eigen-score calculation over sampled response embeddings plus feature clipping in order to control overconfidence. By combining eigen-analysis of hidden layer features with token-level uncertainty, the system LLM-Check [10] works for closed models. Both methods address the question of designing the model either transparent (white-box) or opaque (black-box).

2.4 Ensemble Systems

Researchers Arzt et al. combine a model-agnostic (word alignment using machine translation and BERT for classifier) and a model-aware (activations, attention) detector, and the combined detector reaches accuracies of 80.6% and 81.7% on the two SHROOM tracks. On the other hand, Grigoriadou and colleagues combine an NLI model with a detector and achieve 77.8% and 79.9% respectively.
Both sets of authors show that blending detectors of varying types outperforms having only one.

2.5 Self-Consistency and Efficiency

Despite requiring repeated generation, self-consistency checking is a highly effective method. In their work, Gao et al. [11] take advantage of overlapping tokens for generations with the same prefix in order to shorten the generation duration. This clever method even resulted in up to 3x faster generation with AUROC being retained to the same level as before.

2.6 Datasets and Black-Box Detection

AutoHall [2] automatically develops model-specific hallucination datasets based on fact-checking corpora. It roughly predicts hallucination prevalence of about 20% to 30% across various models. Besides that, it also presents a zero-resource self-contradiction detector that surpasses its baseline counterparts in performance.

2.7 Limits and Applied Systems

In their paper [13], Sinha and co-authors show how embedding-based detection methods are totally ineffective at recognizing true hallucinations resulting from RLHF-aligned models.

The reason lies in the fact that the most difficult hallucinations are semantically extremely similar to loyal responses, which is why people get the semantic illusion. Keerthi et al. [12] have developed a real-time Streamlit app integrating evidence retrieval, semantic similarity measures as well as self-consistency. They claim hybrid methods such as theirs beat single methods but also confess some issues are not yet settled regarding reliability of evidence, speed, and evaluation of benchmarks.

2.8 Positioning of Our Work

The literature points to the fact that sentence similarity detection is cheap but fragile (e.g., [4], [5], [13]). Internal-state methods, on the other hand, are powerful but require model access [9], [10]. Ensembles are effective but multiply components [6], [7]. And self-consistency increases cost [11], [12].

Table 2.1: Summary of Related Work

Authors / System	Category	Key Idea	Main Findings
Bestetti et al.	Survey	Systematic evaluation of methods for hallucination detection in large language models	Highlights the differences between factual hallucination and faithful hallucination and categorizes the various detection methods according to the level of detail.
Cao et al. (AutoHall)	Dataset + black-box detection	How to create domain-specific hallucination datasets automatically by using fact-checking corpora as a basis	Approximately 20 to 30 percent hallucinated content; the self-contradiction detection model exceeds baseline performance.
Mickus et al. (SHROOM)	Shared-task benchmark	Binary classification for overgeneration errors, model-agnostic and model-aware methods	A standardized comparison of detection systems at SemEval-2024
Iordanidou et al. (DUTh)	Sentence similarity	Identify sentence categories using pre-trained encoders such as LaBSE, T5, and DistilUSE	The lack of annotated data can be compensated by synthetic labels.
Bendahman et al. (IRIT)	Embedding analysis	Exploring hallucination sensitivity of embeddings through diagnosis	Performance mainly relies on the selection of the encoder, in addition, the kind of similarity measure employed also plays a role.
Arzt et al. (TU Wien)	Ensemble	Combining model-agnostic and model-aware detectors through fusion	model-aware 80.6%, model-agnostic 81.7% (accuracy)
Grigoriadou et al. (AILS-NTUA)	Ensemble	Fine-tuned detector and NLI model combined	Accuracy came in at 77.8% and 79.9%. Close enough to the competition leaders to matter.

Dale et al.	Internal signals (MT)	Contributions by source help to identify translations which are not connected with the original one	It increases performance by 2 times for serious hallucinations; cross-lingual similarity gets even better
Chen et al. (INSIDE)	Internal states	EigenScore on embedding of responses + limiting of features with clipping	High detection rates were observed on LLaMA, OPT, and Falcon without decoded text
Sriramanan et al. (LLM-Check)	Internal states + uncertainty	Hidden states, eigenvalues, and token uncertainty	Closed models perform well; the approach can capture various forms of hallucinations which are heterogeneous in nature.
Gao et al. (DMP)	Self-consistency efficiency	Exploit shared prefix tokens to perform selective inference	3x speedup without losing AUROC
Keerthi et al.	Applied hybrid System	Pull the evidence, run semantic similarity, then check self-consistency. All in Streamlit.	The hybrid approach outperforms single methods; unresolved questions in evidence reliability, latency, and evaluation
Sinha (Semantic Illusion)	Certified analysis	Conformal forecasting method that guarantees coverages of the finite-sample	Hallucinations generated by RLHF-aligned models can disrupt embedding methods.

Our 13-stage RAG pipeline is inspired by the results mentioned above, claim-level NLI verification based on evidence retrieved rather than embeddings alone [13], combined model-agnostic & model-aware signals [6], [7], calibrated confidence, and evaluation on standard benchmarks [2], [3].

Chapter 3: Research Methodology

3.1 Research Design

The project basically adopts a design-and-evaluation approach. A complete hallucination-detection and fact-verification pipeline for retrieval-augmented generation (RAG) systems is developed, implemented, and evaluated from start to finish. The pipeline consists of thirteen stages and is mapped one-to-one to the architecture diagram of the project. Every stage is implemented as an independently testable software module. What this design is fundamentally based on is the idea that one should not limit hallucination suppression to just putting a single detector at the language model's backend, but rather a multi-step process which first detects, then verifies each claim using retrieved evidence and, when the latter fails, resynthesis, and when it is impossible to trust verification, then refraining.

To put this multilevel approach down to the literature, one of the reasons was a recent systematic review of 50 studies. Published between 2023 and 2025, this review pointed out that two types of hallucination (factuality vs. faithfulness) and two types of model access (white-box vs. black-box) are the two ways the field is split on. It also mentions

LLM-as-a-judge, knowledge graph techniques, external fact-checking, and some others as the top methods used for black-box scenarios.[1]
From the systems point of view, previous work on system prototypes has combined LLM querying, external evidence retrieval, sentence-embedding verification, self-consistency checking, and an interactive interface into a single decision pipeline [2]. This project, in fact, expands the idea in the earlier work to become a complete 13-stage architecture with calibrated confidence estimation, hybrid retrieval, and a three-way decision gate. Each component in the pipeline actually loads a real HuggingFace checkpoint.
Also, for the case when the model cannot be retrieved, the system will explicitly raise an error. There is neither a mock nor a simulated mode in the production code. This choice reflects a methodological and practical stance: it ensures that all the results presented come from actual models, not from simplified stand-in models.

3.2 Overall System Architecture

Figure 3.1 shows the entire flow diagram. The stages are like a chain linking the user question to an assessable, intelligible answer. Besides listing the stages as a straight series, these two figures reveal three flow-control elements which are quite essential. First, the calibration artifact created offline for the confidence module is the Stage 4 input. Second, the persistent retrieval index continues to the Stage 6 feed. Third, the end verification gate in Stage 11 has three outcomes: a PASS decision brings the execution forward, while a REGEN one sends us back to the regeneration phase guided by evidence and, finally, an ABSTAIN decision just says that they do not want to answer a question.
It is a single visual language used across the pictures which is coloured by phase: blue for query understanding (1, 2), green for earlier warning mechanisms (3, 4), amber for claim verification (5~9), red for recovery (10 and 11), and Violet for delivery and assessment (12 and 13). In every subsection under a stage, the flowchart diagram at the beginning is a visual representation for the stage and at the bottom of a stage, the failure behaviour is mentioned as the stage closes. The reason is, when the pipeline breaks the checkpoint failure, error is generated rather than it getting degraded silently.

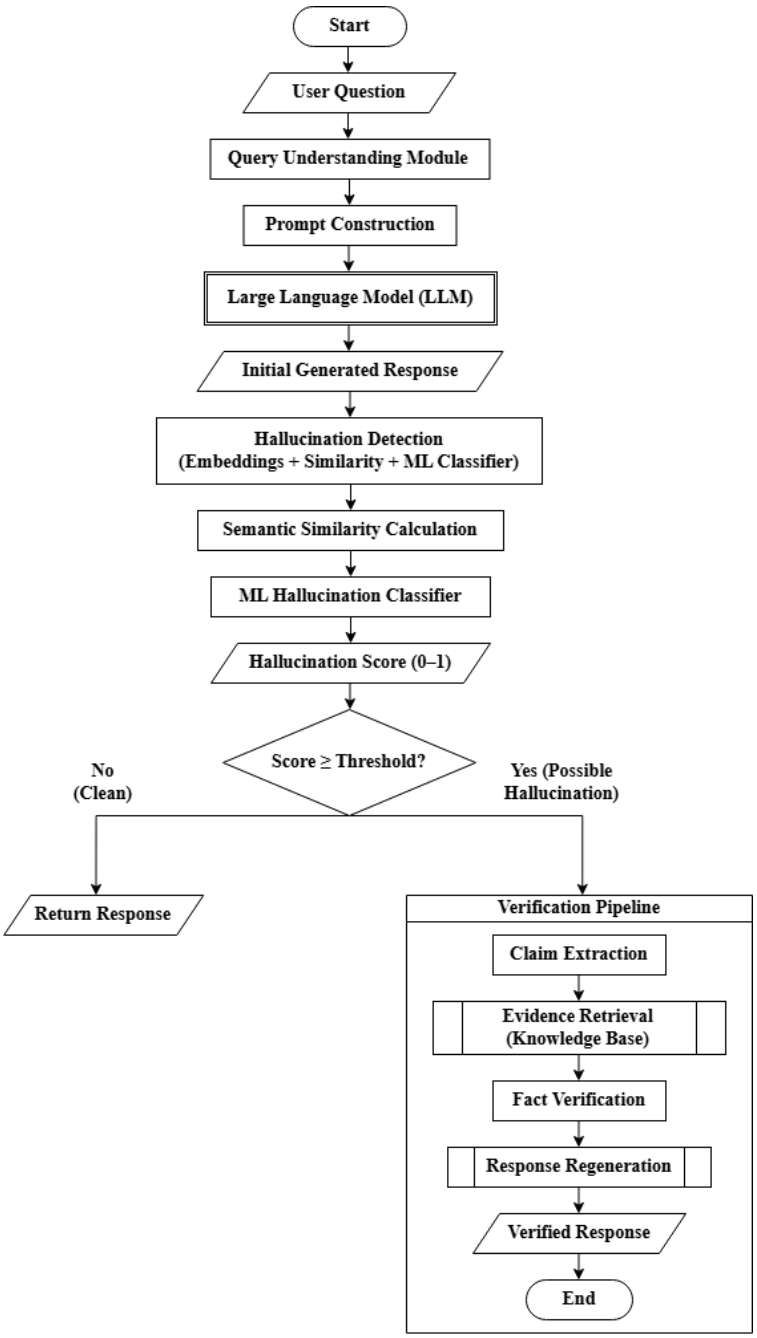


Figure 3.1: Working Procedure of the Proposed Hallucination Detection and Mitigation System for LLM-Generated Responses.

3.2.1 Stage 1: Query Understanding

Figure 3.2 shows what happens in Stage 1 processing. First the user question is passed on to two concurrent entity extractors; one of them is a zero-shot Named Entity Recognition model (GLiNER) and the other one is a spaCy statistical NER model. The entity outputs obtained from these extractors are joined to form a single entity set listing persons, places, organizations and domain terms. Based on this, Llama 3. 1 takes care of querying and re-writing. As a result, it gives back the entity list together with a normalized and a retrieval-friendly query which was previously the user question. It is worth nothing that it is exactly these entities which the fact checking stages after will be looking to protect. Should

either entityRecognition model not be properly loaded, the stage will simply raise an error message without ever letting the incomplete entity set slip.

Figure 3.2:

3.2.2 Stage 2: Grounded Generation

Stage 2 workflow is presented in Figure 3.3. The normalised query is handled by the configured generator, which is the Llama 3.1 8B Instruct model. Qwen3-8B is merely listed as one option that can be switched via the central configuration file without affecting the code. At this generation stage, it is quite on purpose that there be no conditioning, as the pipeline does not require the answer to be a right one right away, but merely looks for how right it is further down the line.

Figure 3.3:

3.2.3 Stage 3: Early Detection

Fig. 3.4 depicts the workflow for Stage 3. BGE-M3 encoder embeds the draft answer, which, together with an XGBoost hallucination classifier, produces the probability of hallucination early, accompanied with a question-answer cosine similarity signal. Logistic-regression is also used here as a fallback. The idea of this screening method is based on an experiment from SHROOM shared task, which stated that hallucinated generations can be discovered with higher reliability through cosine threshold applied for sentence embedding similarity rather than by prompting an LLM for its opinions [3]. Since this preliminary filtering is performed before retrieving and verifying through expensive methods, the number of downstream computations incurred by a response depends on it. In consequence, it is this screen that mainly determines the efficiency of the whole pipeline.

Figure 3.4:

3.2.4 Stage 4: Confidence Assessment

Figure 3.5 displays the working pattern of Stage 4. The raw early score is calibrated by the EEDC module via two techniques Platt scaling and temperature scaling that result in a confidence score that is in range of zero to one and is the input of both score fusion and of the routing logic of the final gate. The calibration step is one of the essential steps and we have a separate calibration script that is able to fit the calibrator and persists it as a versioned artefact. The approach is developed to directly address a documented failure mode which has been reported to the literature: uncalibrated heuristic similarity scores may produce reliable results on synthetic hallucinations but when it comes to realistic hallucinations produced by instruction-tuned models these heuristic scores may fail in a catastrophic way. Calibration techniques with coverage guarantees have been proposed exactly to that end and to turn heuristic scores into decisions [4].

Figure 3.5:

3.2.5 Stage 5: Atomic Claim Extraction

One of the processes in Figure 3.6 is decomposing a response into individual claims by first a two-phase pipeline. A rule-based parser applies splitting by punctuation, and connector and clause boundary rules, which is followed by a splitting step based on an LLM model, here Llama 3.1. This latter step is needed to resolve the remaining complex sentences. The claims from this process,

$$c_1, c_2, \ldots, c_n$$

will serve as the self-verifying units of the whole retrieval and verification system.
In fact, decomposing the generated response first then proceeding with fact-checking is a widely-adopted approach which corresponds to the fact-extraction-verification paradigm most common in knowledge detection papers [1].

Figure 3.6:

3.2.6 Stage 6: Hybrid Retrieval

Figure 3.7 illustrates the process for Stage 6.
The first part shows how the evidence candidates are extracted from dense retrieval (BGE-M3) and sparse retrieval (BM25) for the query and each one of the atomic statements. The two ranked lists are then combined using the reciprocal rank fusion technique.

Besides, the index only required one-time creation from a user-provided corpus in JSONLines format, and along with its metadata, it was subsequently persisted. Retrieval followed by verification is a concept of grounding, as per grounding in formalism by SHROOM through a shared task, where the accuracy of the generated hypothesis in the context of the given reference texts is the evaluation criterion [5].

Hybrid retrieval was the methodological choice to cover failure modes complementary to that of the two types of retrievers: dense retrieval deals with paraphrase quite well whereas sparse retrieval keeps it on par with rare exact terms.

Figure 3.7:

3.2.7 Stage 7: Evidence Reranking

Figure 3.8 displays the sequence of actions in Stage 7. The list of retrieved candidates is resorted by BGE-Reranker-v2 cross-encoder which assesses each claim-evidence pair together rather than separately. Claim-dependent sets of best-ranked passages of evidence, the size of which depends on a user-defined parameter, are forwarded to verification. Cross-encoderbased reranking leads to narrowing down the evidence set so that the entailment phase operates solely on the highly relevant passages.

Figure 3.8:

3.2.8 Stage 8: NLI Verification

Figure 3.9 illustrates the process flow during Stage 8. Each generated claim is compared to its reranked set of evidence sentences by a multilingual natural language inference model (mDeBERTa-v3, XNLI-trained) to come up with a judgment on entailment, contradiction, or neutrality for each claim. Employing the NLI models as hallucination checkers has a solid tradition in the SHROOM literature [6], where NLI-based approaches performed very well in two kinds of tasks, namely the model-aware and the model-agnostic ones. At this step, if a claim can only be inferred from and is supported by the evidence, then it is regarded as verified. If the evidence contradicts the claim, or the relationship is neutral, then the claim remains untested and is therefore, not valid.

Figure 3.9:

3.2.9 Stage 9: Confidence Fusion

Workflow of Stage 9 is visualized in Figure 3.10. Three signal streams converge in Stage 9: the early-detection score from Stage 3, the calibrated confidence from Stage 4, and the

per-claim entailment verdicts from Stage 8. The overall system confidence and the table of the decisions are obtained through the multi-score fusion weighted module. These two outputs together are inputs for the regeneration and the final gate. In SHROOM, systems have found the strongest of the documented methods in combining different detectors through ensemble techniques like voting, probability fusion, and metamodels. For example, these systems have achieved a range of 80.6 to 81.7 percent accuracy by integrating model-agnostic and model-aware signals [7] and 77.8 to 79.9 percent through the combination of a fine-tuned classifier with an NLI model [8]. The fusion weights in the described pipeline are controlled through the configuration files and therefore enable the ablation study as to which signals influence the final decision.

Figure 3.10:

3.2.10 Stage 10: Evidence-Guided Regeneration

Stage 10 workflow is outlined in Figure 3.11. If claims are not verified, the generator will be again triggered in a manner guided by evidence to ensure that new generation will only use verified evidence. It enables the detect-then-rewrite approach to be executed, where flagged items are regenerated and the alternatives are reordered by the quality metric. This is a well-known technique proved to be effective against the generation of hallucinations during testing in MT [9]. Thus, it represents the very first of the two pipeline steps designed to restore a document upon failure.

Figure 3.11:

3.2.11 Stage 11: Final Verification Gate

Figure 3.12 illustrates the workflow diagram for Stage 11.
The final answer, fused confidence, and verdict table go through a three-ways gate. In case of a PASS result, the verified answer is forwarded for rendering the response; in case of a REGEN result, the system gets back to Stage 10 for a new evidencebased attempt which can also be limited by defining maximum retries; and if a ABSTAIN result is obtained, it means that the machine cannot be certain about the content and therefore the answer is not delivered. Choosing not to respond is a methodological choice made on purpose: as deep semantic checks are still not able to distinguish a hallucination that a fluent speaker would make from a faithful answer most of the time, it is better to admit the uncertainty than to show content which has not been checked, especially after the certifiedlimit analysis of embed-based detection [4].

Figure 3.12:

3.2.12 Stage 12: Explainable Response

Figure 3.13 provides a schematic representation of how Stage 12 operates. Each verified response generated by the system is accompanied by a complete history (provenance) that not only includes the final answer but also: a breakdown of judgments per claim, references to supporting documentation, and logs of all thirteen steps of the processing pipeline. The output is available in different formats for easy access: JSON is the format of choice for developers, Markdown is used to compose documents, and HTML is the format of the user interface available on the web. We consider the ability to provide explanations as a core feature of any system design: users should be able to review the evidence against which each claim was verified this transparency of information is in fact one characteristic of successful prior system-level hallucination checkers [2].

Figure 3.13:

3.2.13 Stage 13: Evaluation Module

The workflow of Stage 13 is diagramed in Figure 3.14. This last stage is responsible for computing a complete metric suite per run and then aggregating the results into one metrics file.

It comprises three groups of metrics: quality of generated texts (BLEU score, ROUGE-L, and BERT Score), quality of detection (precision, recall, and F1, ROC-AUC, and the three headline rates HRR, CCR, and VSR), and quality of retrieval (MRR and nDCG). Then the metrics are compared across datasets of synthetic, HaluEval, and TruthfulQA sets.

The idea is to present the generation-overlap metrics (like BLEU, ROUGE, etc.) and detection-level rates side by side. This dual evaluation approach is already used in the hallucination-detection community and it is common to mention detection scores like AUROCs while also referring to text-overlap correctness measures. [10]

Figure 3.14 here:

3.3 Positioning Relative to Internal-State Detection Methods

A carefully considered scope defines the methodology: the pipeline is black-box during detection. It does not rely on getting to know inner working details of the generator, which means it is still useful for generators that offer only API access. This classification positions the system in the black-box branch of the field's taxonomy [1] and, simultaneously, it serves as a complement to the white-box family. The white-box family includes hallucination detection methods: through eigen-analysis of hidden-state covariance with test-time feature clipping [10], from attention maps, hidden activations, and logit signals [11], with speedups over consistency sampling of more than two orders of magnitude, as well as from the accelerated self-consistency sampling methods via selective inference and annealed decoding [12]. Without model internals, the early-detection classifier of Stage 3 simulates the role of these internal signals in white-box systems. A logical follow-up would be the experiment that swaps or enhances Stage 3 with such internal-state scorers when the generator is a local one.

3.4 Models and Technical Stack

We rely on computationally significant components all of which are pre-trained models from the HuggingFace ecosystem. In the case of entity recognition, we join GLiNER with spaCy. A single model: the Llama 3.1 8B Instruct is multi-purpose it supports query rewriting, grounded generation, claim splitting, and evidence-guided regeneration. When necessary, Qwen3-8B is used as an alternative generator.
Two stages in our workflow require BGE-M3 embeddings. The initial detection and the dense retrieval are the stages.

Early hallucination check is done via an XGBoost gradient-boosted decision-tree classifier.
BM25 handles the sparse retrieval part, the BGE-Reranker-v2-M3 does the cross-encoder reranking part, and the multilingual mDeBERTa-v3 is responsible for the final NLI verification. Platt scaling with temperature scaling is a combination used for confidence calibration of the model.

In terms of runtime, for standard Python models, we need Python 3.10 to 3.12 and if real models are required then one will have to install an NVIDIA GPU with about 16 GB of memory, and Llama checkpoints will be retrieved behind a HuggingFace access token.

Our config is organized in a way we call 'single source of truth': all the hyperparameters for your model are kept in a single YAML configuration file. It has one block per step plus a top-level model registry and any value can be changed to one of our environment variables with the prefix. Such a configuration rule is what allows for conducting thorough and systematic ablation experiments without resorting to code change.

3.5 Datasets and Experimental Data

The evaluation is based on the combination of three sources of data. First of all, we used a synthetic question-answering task set that was built-in for the purpose of internal testing, along with an overall evaluation. Secondly, two public benchmarks that are distributed with the project in JSON Lines format constitute the next source of material. Both the HaluEval question-answering section and TruthfulQA have been subjected to the identical evaluation programs. As a known stress test for detection models, HaluEval includes a set of realistic, fluent hallucinations generated by instruction-tuned models instead of artificially created ones [4]. Designing with two benchmarks allows SHROOM to present results from its two tracks in a different setting. Instead of just having a combined performance result, each separate benchmark will give an insight into the results [5].

In addition, user-provided data that makes up the knowledge base for retrieval is the third source of data. Cross-dataset comparison has been made such that it can be a major kind of experiment as well. Once the benchmark results of a specific system have been generated, the system automatically provides cross-comparisons next to each other in a column showing differences, and for every particular query. A smoke-test run that features dataset selection per-dataset is also an option to do a limited number of runs in the same configuration.

When model-specific annotations are necessary besides those benchmarks, one way to do so is with the model-specific hallucination dataset AutoHall created from other fact-check datasets which serves zero-data labeling [13].

3.6 Evaluation Methodology

Evaluation is done on two levels.
The base level evaluation combines a series of single-query metrics to a whole pipeline-level average, but the complete evaluation level also includes the metrics for identification, i.e., precision, recall, and F1, as well as ROC-AUC; there are the three headline detection rates, namely: hallucination reduction rate (HRR), correction coverage rate (CCR), verification success rate (VSR). In total, it is 10 types of metrics that the complete evaluation covers, and checking the metrics file's aggregate block after each run proves to be an effective way of ensuring the correctness of the aggregated performance figures.

The five-step testing guideline comprises: a) the test suite runs fully, b) single-command-line query returns the entire record including answer, per-claim verdicts, evidence, confidence, decision, and step traces, c), the REST API serves up query requests properly, d) the web front-end starts live mode with every checkpoint loaded, and e) evaluation runs must output all ten metric categories without missing any data entry. The confidence calibration takes place before the ultimate detection experiments in order that the confidence-dependent metrics are all calculated based on the calibrated scores.

3.7 System Delivery and Reproducibility

The delivery comes via three different interfaces which use one shared pipeline module underneath: there is a command-line interface which runs inference, evaluation, calibration, and reports; then, a REST service that reveals health condition, indexing, query, metrics,

and structural information; and finally, a front-end based on a browser that adheres to the deployable-interface pattern of earlier hallucination-checking models [2]. The front-end shows a query interface, verdicts of the claims, a decision badge, dataset-aware accuracy and evaluation panels, a cross-dataset comparison card, and the dedication evaluation dashboard with the ability to drill down per query. In case there is some missing component not loading its checkpoint, the live-or-degraded status indicator at the top of the page will let you know so that a degraded run will never be mistaken for a healthy run.

Reproducibility is based on three mechanisms. Firstly, there is an automated test suite with 31 tests which replace the model slots with lightweight in-memory doubles during testing so that pipeline logic can be verified without requiring model downloads and offline. Production code, however, always calls real models. Secondly, the component-level failures are reported as degraded states without being silently handled. Finally, the full process of building metrics from an initial repo clone to a populated repo is captured as a single ordered command sequence which serves as the thesis replication recipe.

3.8 Project Structure and Modularity

The codebase corresponds directly with the architecture diagram, and modules for each pipeline stage are in an individual steps package. Model wrappers, retrieval implementations (FAISS, BM25, and reciprocal-rank-fusion hybrid), and utility components (chunker, claim extractor, data loader) are placed in their own packages. A REST service and a web front-end constitute the interface layer. Moreover, command-line entry points are centralized in a scripts directory.

Data types that are shared such as evidence and claim records are the contract that binds the stages. This one-module-per-step discipline is the foundation of the method that enables the ablation studies of Section 3.6 through the fact that with just configuration change any stage can be reconfigured, bypassed, or exchanged. This is exactly the scenario that is shown by the per-stage workflow diagrams illustrated in Figures 3.2 through 3.14.